Travlr Getaways

CS 465 Project Software Design Document

Version 1.0

Table of Contents

CS 465 Project Software Design Document 1
Table of Contents 2
Document Revision History 2
Instructions 2
Executive Summary 3
Design Constraints 3
System Architecture View 3
Component Diagram 4
Sequence Diagram 5
Class Diagram 6
API Endpoints 7
The User Interface 7

Document Revision History

Version	Date	Author	Comments
1.0	02/01/2026	Jose Morales Egipciaco	Draft for Milestone One
2.0	02/15/2026	Jose Morales Egipciaco	Milestone Two – Add Sequence and Class Diagram

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Travlr Getaways web application is built using the MEAN stack (MongoDB, Express.js, Angular, and Node.js), providing a comprehensive full-stack solution for travel booking and administration. The architecture consists of two primary components: a customer-facing website and an administrator Single-Page Application (SPA).

The customer-facing website utilizes server-side rendering with Express.js and Handlebars templates to deliver travel listings, trip details, and promotional content. This traditional multi-page approach ensures optimal SEO performance and fast initial page loads for public users browsing vacation packages.

The administrator SPA is built with Angular, providing a dynamic, responsive interface for managing trip inventory. This SPA communicates with a RESTful API backend to perform CRUD (Create, Read, Update, Delete) operations on trip data stored in MongoDB. The SPA architecture enables administrators to add, edit, and remove trips without page reloads, creating a seamless user experience similar to desktop applications. Authentication and authorization mechanisms ensure that only authorized users can access administrative functions.

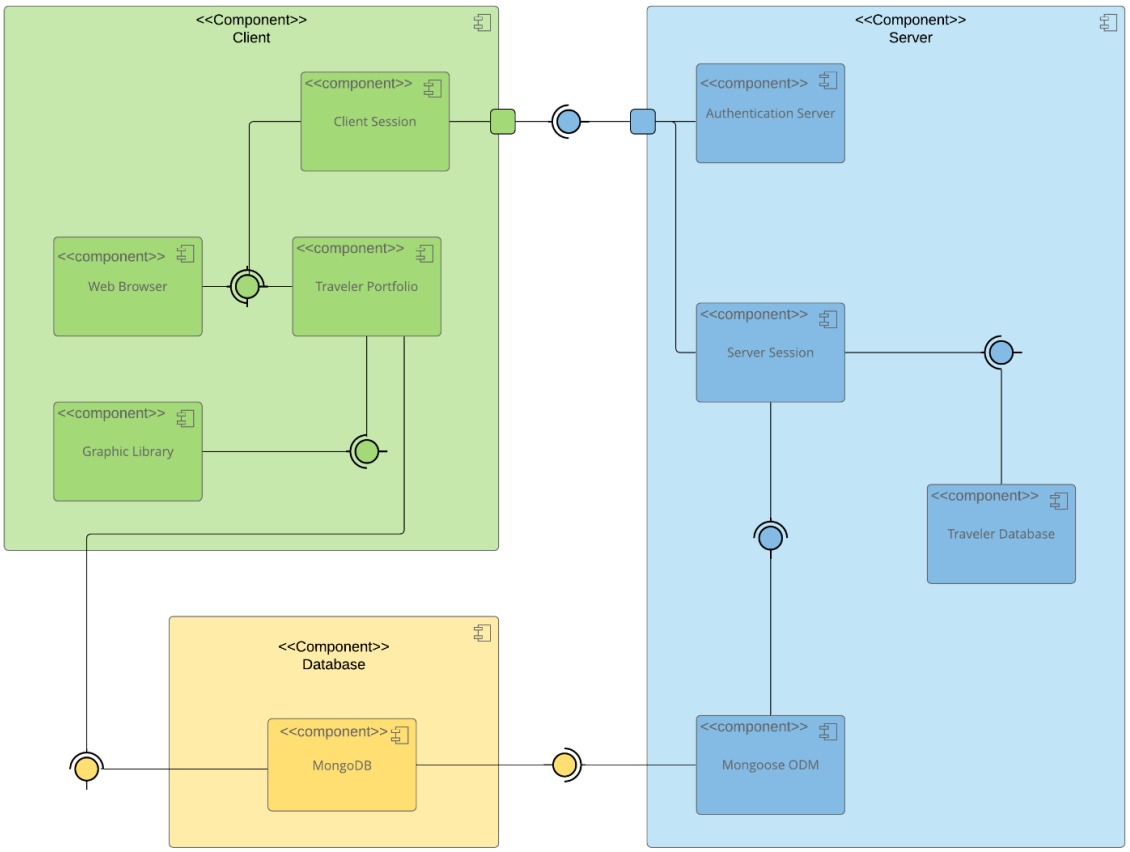
Design Constraints

Several design constraints impact on the development of the Travlr Getaways application.

> First, the MEAN stack architecture requires JavaScript proficiency across the entire development team, as both frontend and backend use JavaScript-based technologies. This creates a unified development environment but limits technology choices.
>
> Second, the application must maintain separate rendering approaches for the public website (server-side with Handlebars) and the admin interface (client-side with Angular). This dual architecture increases complexity but optimizes each component for its specific use case—SEO for the public site and interactivity for the admin panel.
>
> Third, security constraints require implementing authentication and authorization mechanisms to protect administrative functions. The RESTful API must validate user credentials and session tokens for all data-modifying operations. Additionally, MongoDB's NoSQL structure requires careful schema design to maintain data integrity without traditional relational database constraints. These security and data management requirements add development time but are essential for a production-ready application.

System Architecture View

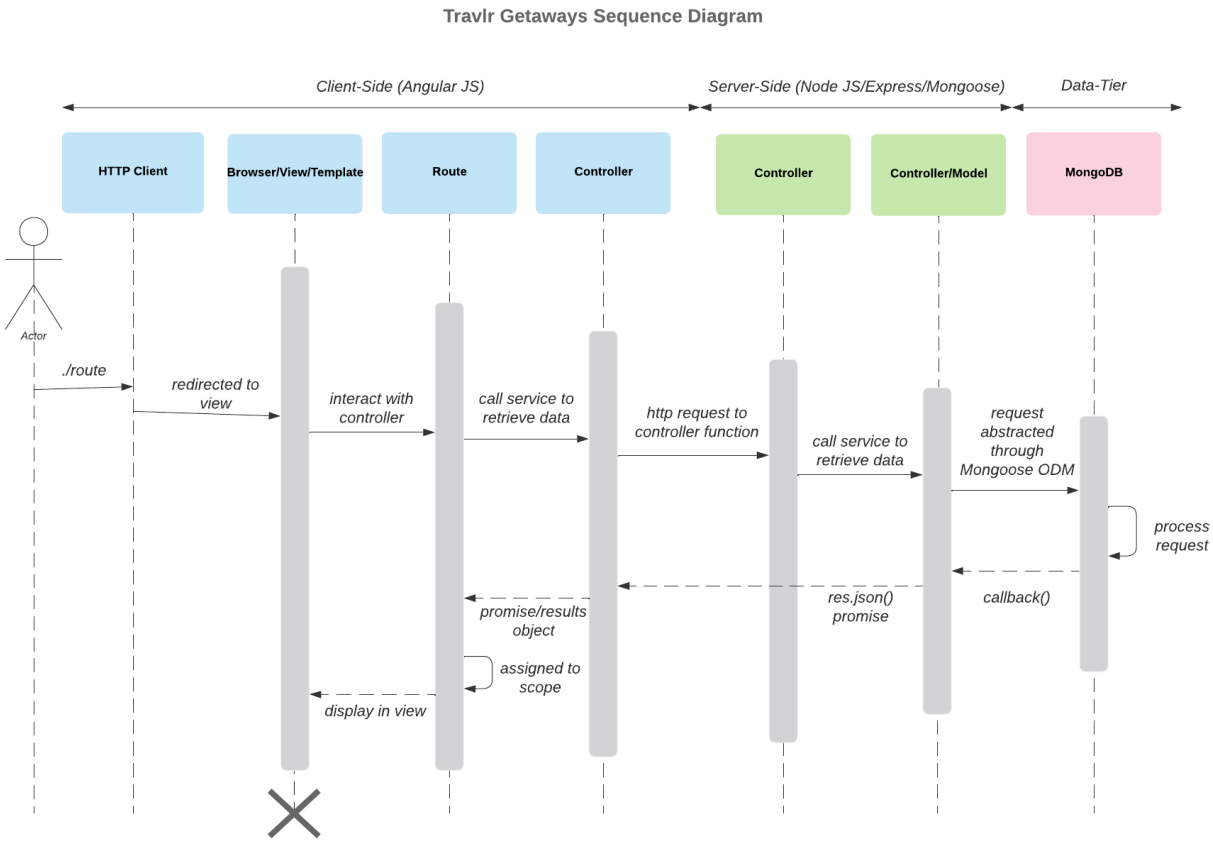
Component Diagram



A text version of the component diagram is available: [CS 465 Full Stack Component Diagram Text Version].

The system architecture consists of three primary tiers working together to deliver the Travlr Getaways application. At the presentation tier, users interact with either the customer-facing website rendered server-side using Handlebars templates or the Angular-based SPA for administrators. These interfaces provide distinct user experiences while accessing the same underlying data.
The application tier is powered by Node.js and Express.js, which handle HTTP requests, routing, and business logic. For the customer website, Express renders Handlebars templates. For the admin SPA, Express exposes RESTful API endpoints that the Angular application consumes to retrieve and update data.
The data tier utilizes MongoDB to store trip information and application data in flexible documents. Mongoose acts as the Object Data Modeling library, allowing the server to interact with the database through structured models. Together, these tiers create a cohesive full-stack system where the frontend communicates with Express, Express processes logic and interacts with MongoDB, and data flows back to update the user interface.

Sequence Diagram

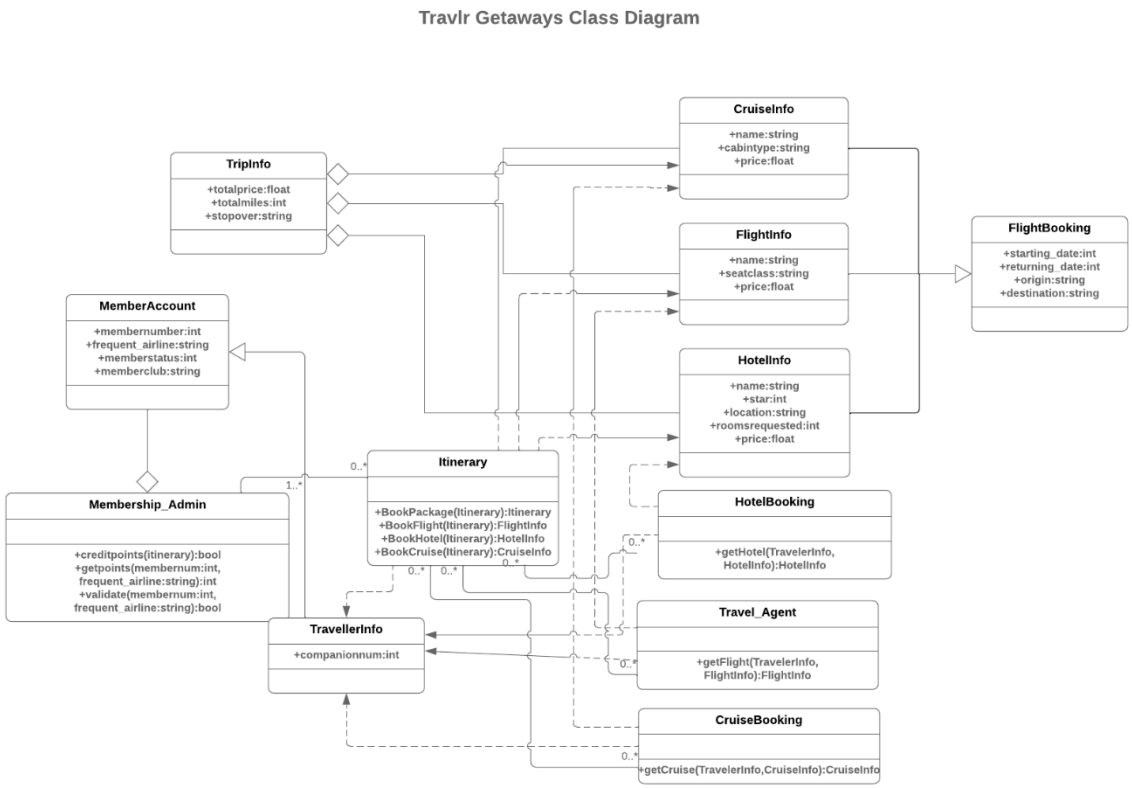


The application flow follows a three-tier interaction pattern. For the customer-facing trips display, the browser sends an HTTP GET request to the Express server at the /travel endpoint. The Express router directs this to the Travlr controller, which queries MongoDB through Mongoose to retrieve trip documents. The database returns the trip data, which the controller passes to the Handlebars view engine for server-side rendering. The rendered HTML is sent back to the browser for display.
For admin authentication, the Angular SPA sends credentials via an HTTP POST request to the /api/login endpoint. Express validates these credentials against stored user data in MongoDB. If valid, the server generates a JSON Web Token (JWT) and returns it to the Angular app, which stores it for subsequent authenticated requests.

Admin CRUD operations follow a similar pattern with added authentication. When an administrator updates a trip, the Angular app sends a PUT request to /api/trips/:tripId with the JWT in the request header. Express middleware validates the token, then the trip controller processes the update request. Mongoose updates the specified document in MongoDB, and the database confirms the operation. The success response flows back through Express to the Angular app, which updates its local state and

refreshes the UI. This separation of concerns across tiers ensures security, maintainability, and scalability of the application.

Class Diagram



The application's class structure includes several key Angular components and backend models. On the frontend, the TripListingComponent displays all available trips by consuming the TripDataService, which handles HTTP communication with the backend API. The TripCardComponent renders individual trip details, accepting trip data as input properties. The AddTripComponent and EditTripComponent manage form inputs for creating and modifying trips, utilizing Angular's reactive forms for validation.
At the backend, the Trip model (defined using Mongoose schema) specifies the structure of trip documents with properties like code, name, length, start, resort, perPerson, image, and description. The TripController class contains methods like tripsList(), tripsAddTrip(), tripsGetCode(), and tripsUpdateTrip() that handle business logic for CRUD operations. The AuthenticationController manages user login and token generation. These classes work together through dependency injection and service patterns—Angular components depend on services for data access, services communicate with Express controllers via HTTP, and controllers interact with Mongoose models to persist data in MongoDB.

API Endpoints

Method	Purpose	URL	Notes
GET	Retrieve list of trips	/api/trips	Returns all trips in JSON format for display in the SPA
GET	Retrieve single trip	/api/trips/:tripCode	Returns a specific trip identified by trip code for editing
POST	Create a new trip	/api/trips	Adds a new trip to the database, requires authentication
PUT	Update an existing trip	/api/trips/:tripCode	Modifies an existing trip, requires authentication
DELETE	Delete a trip	/api/trips/:tripCode	Removes a trip from the database; requires authentication
POST	User login	/api/login	Authenticates admin user and returns JWT token

The User Interface

Screenshots of the Angular SPA (trip creation, edit, and update screens) * will be added in a later milestone once the admin interface is fully implemented.

The Angular project structure differs significantly from the Express structure. While Express follows a traditional MVC pattern with routes, controllers, and views organized in separate directories, Angular uses a component-based architecture where each feature is encapsulated in its own module with components, services, and templates. The Angular SPA resides in the app_admin folder with a src/app directory containing components like trip-listing, trip-card, add-trip, and edit-trip, along with services like trip-data.service for API communication.

The SPA provides significantly richer functionality than traditional web applications. Rather than full page reloads, Angular updates only the necessary portions of the DOM, creating a fluid, desktop-like experience. The SPA maintains application state in memory, enabling instant navigation between views and real-time form validation. Two-way data binding automatically synchronizes the model and view, so changes in form inputs immediately reflect in the component's data model and vice versa. This reactive approach eliminates the need for manual DOM manipulation and provides immediate user feedback.

Testing the SPA involves multiple approaches. During development, browser developer tools monitor network requests to verify correct API endpoints are being called with proper HTTP methods and payloads. The Network tab shows GET requests to /api/trips returning trip arrays and PUT requests to

/api/trips/:tripCode sending updated trip data. Angular's development server provides live reload, allowing immediate verification of changes. For systematic testing, developers can use Angular's built-in testing framework with Jasmine and Karma to write unit tests for components and services. Integration testing verifies that clicking "Edit" loads trip data into the form, modifying fields updates the model, and submitting sends the correct PUT request. Successful API communication is confirmed when the database reflects changes and the UI updates accordingly without page refreshes.